🤖 Build → 👤 Simulation → 💬 Dialogs → 📊 Evaluation

SDialog: A Python Toolkit for End-to-End Agent Building, User Simulation, Dialog Generation, and Evaluation

Sergio Burdisso, Séverin Baroudi, Yanis Labrak, David Grunert, Pawel Cyrta, Yiyang Chen, Srikanth Madikeri, Esaú Villatoro-Tello, Ricard Marxer, and Petr Motlicek

🔗 **A unified workflow bridging agent construction → user simulation → dialog generation → evaluation → mechanistic analysis.**

⚠️ Fragmented Ecosystem

Current conversational AI workflows remain disconnected:

- 🗄️ **Data:** Heterogeneous dataset formats.
- ⚙️ **Generation:** Limited controllability in synthetic pipelines.
- 📊 **Evaluation:** Metrics detached from model internals.
- 🔬 **Analysis:** Interpretability tools not dialog-aware.

🧩 A Unified Framework

Dialog-Centric Architecture
SDialog provides a structured `Dialog` abstraction enabling reproducible experimentation across the full conversational pipeline.

🧰 System Components

📦 **Dialog Core Object:** Metadata-rich representation with lineage tracking and multi-format I/O (JSON, CSV).

👤 **Personas & 🤖 Agents:** Structured personas (role, style, demographics) and tool-augmented LLM agents with memory and optional reasoning mode.

🎛️ **Orchestration Layer:** Composable flow control (length constraints, instruction injection, adaptive policies): `agent = agent | length_orch | reflex_orch`

📋 **Evaluation Framework:**

- Linguistic, semantic, and flow-based metrics.
- LLM-as-a-Judge for realism, refusal behavior, and persona adherence.
- Functional tool-sequence validation.
- Cross-model comparator benchmarking.

🧠 **Mechanistic Interpretability:** Token-level activation inspection and causal steering via targeted intervention: `agent = agent | inspector - refusal_vector`

🎙️ **Audio Extension:** TTS with acoustic simulation (room geometry, microphone placement) for realistic speech datasets.

📊 System Demonstration

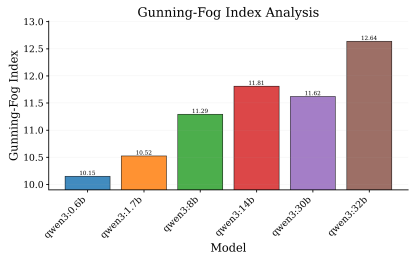
Scenario: Call-center support agent with three callable tools.
Task: Comparative evaluation of Qwen3 models (0.6B–14B) on functional correctness and linguistic complexity.

📈 **Findings:**

- **Functional Performance:** Larger models do not consistently improve constrained workflows. The 8B model achieved strong verification sensitivity (0.97) with correct tool sequencing (0.83), outperforming 14B under strict conditions.
- **Linguistic Complexity:** Gunning Fog index increases with scale (10.1 → 11.8), despite identical prompts.

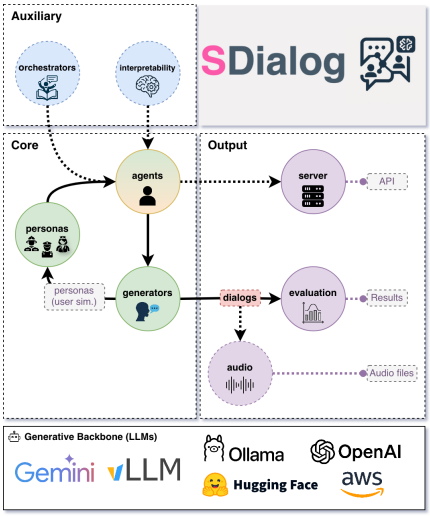
Model	Case A (Verification Required)		Case B (No Verification)	
	Ask-Verify	Tools-OK	Ask-Verify	Tools-OK
qwen3:0.6b	0.82	0.01	0.63	0.09
qwen3:1.7b	0.33	0.00	0.18	0.00
qwen3:8b	0.97	0.83	0.38	0.82
qwen3:14b	**1.00**	0.56	**0.06**	**0.93**
qwen3:30b	**1.00**	**0.96**	0.25	0.89
qwen3:32b	**1.00**	0.88	0.23	0.89

Table 1: Functional correctness across Qwen3 sizes. Metrics show proportion of dialogs where agent asks for verification (Ask-Verify) and correctly follows target tool sequences (Tools-OK).



Designed for reproducible dialog research and controlled system experimentation.

🧱 Architecture



💻 In Practice

```python
import sdialog
from sdialog import Context
from sdialog.agents import Agent
from sdialog.personas import SupportAgent, Customer
from sdialog.orchestrators import SimpleReflexOrchestrator

sdialog.config.llm("openai:gpt-4.1", temperature=1)
# sdialog.config.llm("ollama:qwen3:14b")  # etc.

# ----------------------------------------------------------------------
support_persona = SupportAgent(name="Ava",
                               politeness="high",
                               communication_style="friendly"
)
customer_persona = Customer(name="Riley",
                            issue="double charge",
                            desired_outcome="refund")
# ----------------------------------------------------------------------
def verify_account(user_id):
    """Verify user account by user id."""
    return {"user_id": user_id, "verified": True}
def refund(amount):
    """Process a refund for the given amount."""
    return {"status": "refunded", "amount": amount}

support_agent = Agent(
  persona=support_persona,
  think=True,
  tools=[verify_account, refund],
  name="Support"
)
simulated_customer = Agent(
  persona=customer_persona,
  first_utterance="Hi!",
  name="Customer"
)
# ----------------------------------------------------------------------
react_refund = SimpleReflexOrchestrator(
  condition=lambda utt: "refund" in utt.lower(),
  instruction="Follow refund policy; verify account, apologize, refund.",
)
support_agent = support_agent | react_refund
# ----------------------------------------------------------------------
web_chat = Context(location="chat",
                   environment="web",
                   circumstances="billing")
for ix in range(3):
  dialog = simulated_customer.dialog_with(support_agent, context=web_chat)
  dialog.to_file(f"dialog_{ix}.json")
# ----------------------------------------------------------------------
support_agent.serve(port=1333)
```

idiap RESEARCH INSTITUTE

Demo | Github | Paper